University of Toronto

Faculty of Applied Science & Engineering

ECE496 Proposal

Project Title	Building an Intelligent Job Scheduler for Optimized Request Assignment
Project Number	PROJ 521
Supervisor	Ali Bereyhi
Administrator	Sarah Oliveira
Prepared By	Housen Zhu Ruoyu Li Yifan Wang Yuan Sui
Date of submission	10/25/2024

1. Executive Summary

The proposal outlines the development of an intelligent job scheduler designed to optimize request assignment in emergency departments, utilizing advanced deep reinforcement learning (DRL) techniques. The project addresses critical challenges such as overcrowding and prolonged waiting times caused by traditional rule-based scheduling methods. By employing cutting-edge algorithms like Deep Q Network (DQN), the system aims to significantly reduce delays when handling requests.

There are three key features for this project, including a chatbot for collecting user information, a forecasting component to predict incoming request loads, and a user-friendly graphical user interface (GUI). Based on that, the system's dynamic scheduling mechanism will adjust in real-time to fluctuations in demand, ensuring efficient resource allocation. Testing will occur in simulated environments, with the final product targeted to enhance teleconsultation processes by reducing average wait times by 9% to 41%.

Moreover, the project is structured around critical milestones, from research and system design to integration and testing. Challenges, such as data availability and algorithm performance, are addressed with contingency plans, including the use of synthetic data and traditional scheduling methods as benchmarks. The budget for the system's development is capped at 400 CAD, and the project adheres to strict data privacy regulations. Ultimately, this intelligent job scheduler is expected to improve the operational efficiency of emergency departments and teleconsultation services significantly.

2. Background and Motivation

Emergency departments are highly complex and often face serious challenges of overcrowding and long patient waiting times. A number of factors contribute to the complexity of patient flow, including patients with varying levels of severity and the availability of medical resources [1]. The unpredictability of patient arrivals and changing resource constraints further exacerbate this complexity, making it difficult for decision makers to effectively manage patient scheduling. Scheduling errors can lead to longer patient waiting times, overcrowding, and potential deterioration in patient health.

For several decades, substantial efforts have been made to resolve the crowding caused by the complexities in emergency departments. Traditionally, this problem is solved by rule-based algorithms (e.g. assigning patients to the next available doctor). However, such algorithms often result in suboptimal performance and long delays due to an inability to adapt to variable patient demand [2]. Recently, advancement in deep reinforcement learning has enabled new algorithms to leverage trends in past data to forecast patient demand and schedule accordingly to significantly reduce wait time. Currently, the state of

art scheduler for teleconsultation utilizes Deep Q Network (DQN) and achieved 9%–41% reduction in average waiting time [3].

Our scheduler will implement other deep reinforcement algorithms such as Deep Deterministic Policy Gradient (DDPG) and Proximal Policy Optimization (PPO) to further increase the performance of the scheduler. Encasing the novel algorithm, our scheduler will include a built-in chatbot for collecting request information, a network prediction component to forecast incoming request loads, and an efficient scheduling system to minimize patient wait times. In addition, our capstone product will include a user-friendly graphical user interface (GUI) to ensure accessibility for all users, thereby enhancing the overall efficiency and effectiveness of patient service interactions.

3. Problem Statement

The project seeks to address the challenge of inefficient request assignment in emergency departments, where traditional rule-based scheduling leads to prolonged patient waiting times and overcrowding [4]. By applying Deep Reinforcement Learning (DRL), our intelligent job scheduler will optimize the scheduling algorithm to enhance the overall efficiency.

4. Project Goal

The goal of this project is to develop a robust and user-friendly job scheduler that significantly leverages Deep Reinforcement Learning to optimize the allocation of incoming requests to medical resources and minimize wait time.

5. Scope of Work

This project is to design an intelligent scheduler for emergency departments that optimizes the allocation of incoming patients and medical resources. Testing and evaluation of our job scheduler will be conducted in the simulated environments (simulated user input test cases). Real-world application and testing fall outside the scope of this project. Our specific scopes of the project are outlined below.

5.1 Algorithm and Model Architecture

The well-designed scheduling system will dynamically adjust strategies to minimize wait times and maximize distribution efficiency. Our original contributions will be our own customized version of DRL models including the design of the reward functions and policies. By significantly reducing delays in handling patient requests, this system ensures real-time efficient resources assignments.

5.2 Load Forecasting Function

To better serve the scheduling optimization, we intend to involve the forecasting model to evaluate incoming request loads. In this case, the scheduler can adjust scheduling priorities actively regarding the predicted incoming load. The original contribution will focus on designing a forecasting algorithm to cater to our system and integrating this forecasting function into the scheduling system to ensure fluent assigning.

5.3 User Interface and Chatbot Design

Our system will introduce a user-friendly chatbot interface to provide a seamless user experience. It will take in user input, such as progress inquiry, then provide real-time updates on patient scheduling. The chatbot architecture will rely on existing web-based GUI frameworks, such as Gradio [5].

5.4 System Context Diagram

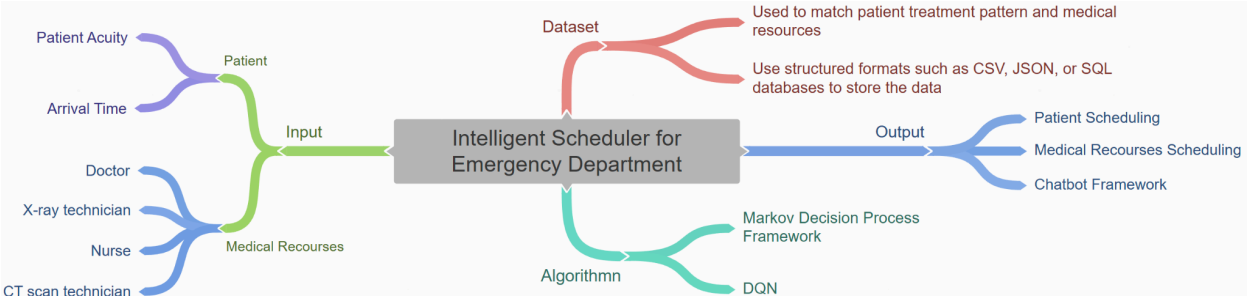


Figure 1: System Context Diagram

6. Requirements Specification

This section specifies the functions, constraints, and objectives of the design for the job scheduler. We use these requirements to assess whether a proposed design meets the criteria. Below is the requirements table:

ID	Project Requirement	Description
1	Allocation of incoming requests to agents.	Functional: the scheduler must allocate the incoming patient requests to medical resources.
1A	DRL-based agent assignment	Constraint: The system must assign incoming requests using a DRL algorithm.

1B	Minimize the average delay experienced by users	Objective: The average delay using DRL should be lower than that of the rule-based algorithm.
1C	Concurrent request handling	Objective: the system should handle at least 1000 concurrent user requests [6].
2	Chatbot for user interaction	Functional: the chatbot should be able to show treatment schedules and handle initial interactions with users.
3	Network load prediction	Functional: th·the system must predict incoming request load, including volume and distribution.
4	User-friendly GUI	Functional: the system must have a GUI.
4A	Minimize the GUI complexity	Objective: the GUI should allow users to perform key tasks in no more than 3 clicks [7].
4B	User interface response time	Objective: the system should operate with a response time of less than 1s per action in UI [8][9].
5	System development cost	Constraint: the cost of developing the system must not exceed 400 CAD.
6	Data privacy compliance	Constraint: the system must comply with all relevant legal and privacy regulations when handling user data.

7. Project Milestones

Milestone 1: Preliminary Research and Strategic Planning (8.1-9.14)

- Define project objectives and success metrics.
- Conduct a comprehensive literature review on existing scheduling algorithms and DRL applications.
    - Identify and summarize key research papers.
    - Analyze the strengths and weaknesses of different approaches.
- Identify key system requirements and constraints for the scheduler.
    - Determine functional and non-functional requirements.
    - Define system constraints and limitations.
- Review and refine objectives and success metrics.

Milestone 2: In-Depth Background Study and System Design (9.15-10.13)

- Conduct an exhaustive study on DRL methodologies, focusing on suitable algorithms (e.g., DQN, PPO).

    - Evaluate the applicability of each algorithm.
    - Document findings and select the most suitable DRL approach.
- Design the overall system architecture, and find out the primary components (DRL scheduler, chatbot, network prediction, GUI).
    - For example, create a system block diagram and define interactions between components.
- Select appropriate technologies and tools for system implementation.
    - Create a workspace to store all the codes, and clarify a set of code styles.

Milestone 3: Data Acquisition and Preprocessing (10.13-10.27)

- Collect historical data on request types, agent performance metrics, and other relevant parameters. Goal: gather and consolidate data.
- Clean and preprocess the data for effective training of the DRL model.
- Finalize the dataset and prepare for model training (training and validation sets).

Milestone 4: Prototype Development and Initial Testing (10.27-11.17)

- Develop a preliminary DRL model.
- Create a basic chatbot to handle initial user queries and gather necessary information.
- Develop a prototype GUI for initial user interaction.

Milestone 5: System Integration and Comprehensive Testing (11.18-12.8)

- Debug code.
- Integrate the DRL model with the chatbot and GUI components.
- Conduct thorough unit testing of individual components.
- Perform extensive integration testing to ensure seamless operation of all system components.

Milestone 6: Advanced Features Implementation and Optimization (1.6-1.26)

- Incorporate advanced DRL techniques to enhance scheduling efficiency.
- Upgrade the chatbot to support more sophisticated interactions.
- Optimize the network load prediction model for better performance.

Milestone 7: Evaluation, Refinement, and Validation (2025 Jan - Feb)

- Conduct rigorous performance testing using various metrics (e.g., average wait time, system load).
- Refine the model based on test results and user feedback.
- Prepare comprehensive technical documentation and user manuals.

Milestone 8: Final Deployment and Formal Presentation (2025 Feb - April)

- Deploy the scheduler in a controlled environment for final testing and validation.
- Compile a detailed project report and prepare a professional presentation.
- Present the project to stakeholders, showcasing key features, results, and benefits.

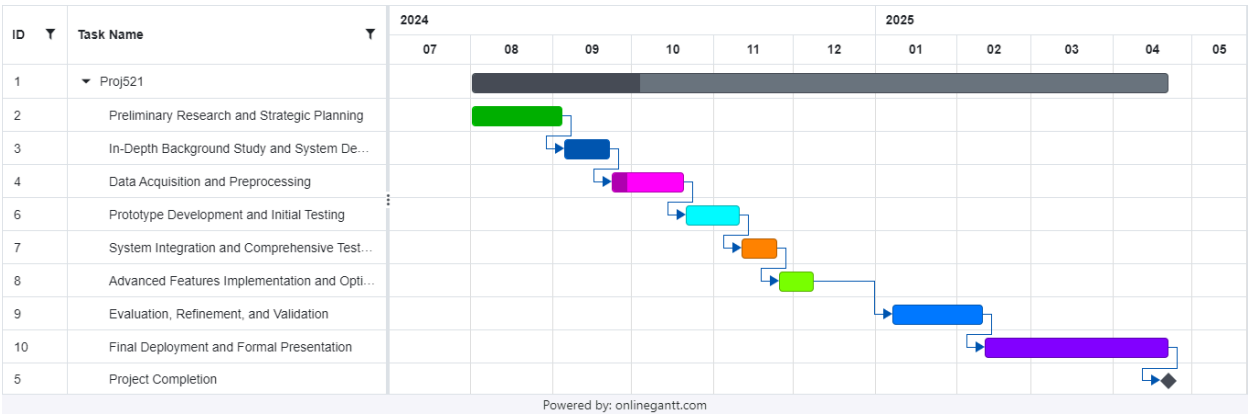


Figure 2: gantt chart of milestones

8. Feasibility Assessment

In this section we first describe the skills, knowledge and resources we need to successfully complete the project. Secondly, we discuss potential risks of the project, where it could fail and how we can prevent those failures.

8.1 Skills, Knowledge, and Resources

The following three tables list out the essential skills, knowledge and resources needed for this project. We also summarize our current progress and plan for obtaining these items.

- Skills: Software (Python) and website development (Gradio) skills are already acquired, as team members are proficient in these areas.
- Knowledge: A thorough review of neural networks is complete, giving us a solid foundation for implementing deep reinforcement learning (DRL). We are still progressing through DRL concepts.
- Resources: We have finished building a preliminary data structure, and we plan to start batch generating customized emergency department data in the next few weeks in order to present better results for the project.

8.2 Risks and Unknowns

Our project plans to approach the problem of medical resources scheduling with DRL. Although DRL shows great performance in many other areas such as robotic control and machine scheduling [10], it is possible that this algorithm fails in a complex environment like an emergency room. Thus, we also plan to implement many other traditional scheduling algorithms to act as benchmark and alternative solutions to the DRL method. Specifically, we also plan to implement the following algorithms [11]:

Algorithm	**Description**
First Come First Serve (FCFS)	Select the patient based on who arrived the earliest
Shortest Remaining Processing Time (SRPT)	Select the patient based on shortest remaining

	process time
Acuity and Waiting Time (AW)	Select patient based on acuity and waiting time
Acuity and Arrival time (AA)	Select patient first based on high acuity level then based on earliest arrival time

9. References

[1] S. Nakstad, I. Guttormsen, and S. Strand-Amundsen, "Norwegian Helicopter Emergency Medical Services (HEMS) early response to the COVID-19 pandemic: an observational study," Scandinavian Journal of Trauma, Resuscitation and Emergency Medicine, vol. 28, no. 1, pp. 1-9, 2020. [Online]. Available: https://doi.org/10.1186/s13049-020-00749-2. (accessed Sep. 23, 2024)

[2] “Online doctor consultations - global: Market forecast,” Statista, https://www.statista.com/outlook/hmo/digital-health/online-doctor-consultations/worldwide (accessed Sep. 16, 2024).

[3] A. Roy, J. L. Pachuau, and A. K. Saha, “An overview of queuing delay and various delay based algorithms in networks - computing,” SpringerLink, https://link.springer.com/article/10.1007/s00607-021-00973-3 (accessed Sep. 16, 2024).

[4] Wenjia Chen, Jinlin Li, “Teleconsultation dynamic scheduling with a deep reinforcement learning approach,” Artificial Intelligence in Medicine, https://www.sciencedirect.com/science/article/pii/S0933365724000484 (accessed Sep. 16, 2024).

[5] “First-come-first-served,” First-Come-First-Served - an overview | ScienceDirect Topics, https://www.sciencedirect.com/topics/computer-science/first-come-first-served (accessed Sep. 16, 2024).

[6] G. Team, “Gradio docs,” Gradio Docs, https://www.gradio.app/docs/gradio/interface (accessed Sep. 16, 2024).

[7] J. Solanki, “How to build a scalable application up to 1 million users on AWS,” Simform, https://www.simform.com/blog/building-scalable-application-aws-platform/ (accessed Sep. 16, 2024).

[8] About The Author UX World UX Passionate who loves to share practical experiences and tips around UX and product design., “3-click rule and usability,” UX Design World, https://uxdworld.com/3-click-rule-and-usability/ (accessed Sep. 16, 2024).

[9] “Usability engineering : Book by Jakob Nielsen,” Nielsen Norman Group, https://www.nngroup.com/books/usability-engineering/ (accessed Sep. 16, 2024).

[10] X. Xia, Z. Tang, B. Zhang, and M. K. Ng, "Distributionally Robust Semi-supervised Learning on Graphs with Self-training," arXiv, 2023. [Online]. Available: https://arxiv.org/abs/2310.03195 (accessed Sep. 16, 2024)

[11] R. Lochlainn et al., "Regulation of ACE2 expression and activity in the gut," Microorganisms, vol. 8, no. 7, p. 1111, 2020. [Online]. Available: https://doi.org/10.3390/microorganisms8071111 (accessed Sep. 16, 2024)